Making Grids

Required packages

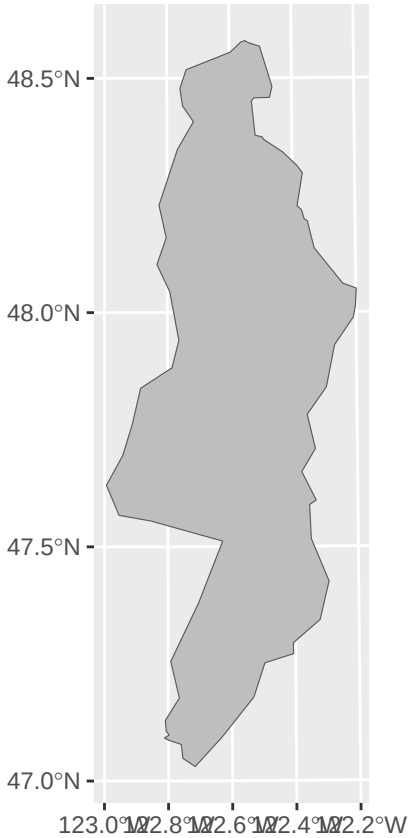
```
library(ggplot2) # plotting
library(sf) # spatial
library(terra) # raster
library(tidyverse)
library(nngeo)
```

Description of Initial Files

- `reduced_area.gpkg` is a hand drawn polygon around the desired study area originally made in **QGIS**. *CRS: 26910 (meters)*

```
studyarea <- st_read("GIS_COVS/GPKG/reduced_area.gpkg")
```

```
ggplot()+
  geom_sf(data = studyarea, fill ="gray")
```

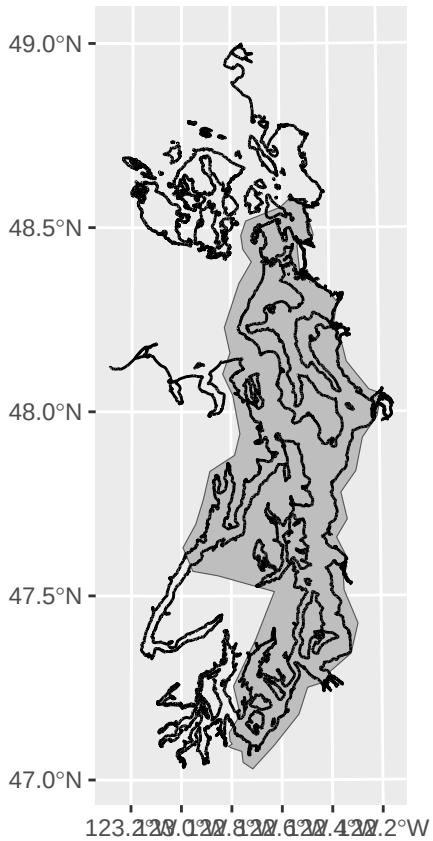


- `Dclass_cut.gpk` is the geopackage for the Dethier Class which was adapted from the WA ShoreZone Inventory in 2014. The html provides further description. It is simply the clipped version within the study in gpkg format of the downloadable file from the link above. The clipping was originally done in *QGIS*. The gpkg is used as the shoreline for clipping purposes. It is later further cropped.

CRS: 26910 (meters)

```
shoreline <- st_read("GIS_COVS/GPKG/Dclass_cut.gpkg")
```

```
ggplot()+
  geom_sf(data = studyarea, fill ="gray")+
  geom_sf(data = shoreline, color = "black")
```



- `SS_Bathymetry.tiff` is the Bathymetric Base Map for the Salish Sea Bioregion by Aquila Flower, 2021. I use it to make a closed polygon shape for other analyses. 3-arc second (approximately 90 meter) spatial resolution. Vertical datum: approximately Mean Sea Level.

CRS 26910

```
ss_bath <- terra::rast("GIS_COVS/SS_Bathymetry/SS_Bathymetry.tif")
```

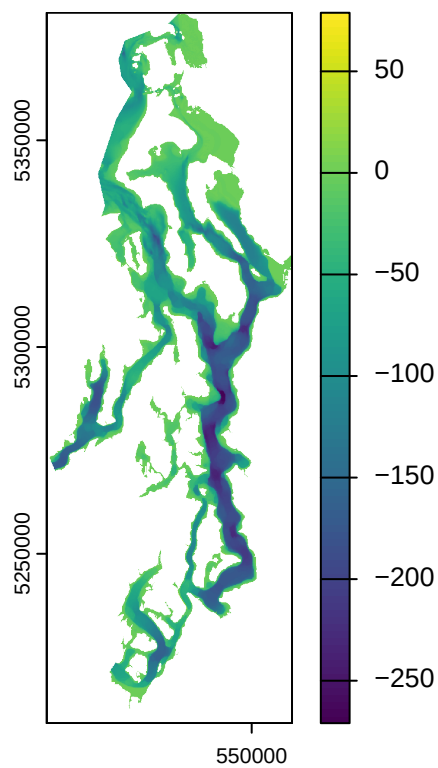
```
ss_bath
```

```
## class       : SpatRaster
## size        : 5277, 4256, 1  (nrow, ncol, nlyr)
## resolution  : 90, 90  (x, y)
## extent      : 289484.9, 672524.9, 5160345, 5635275  (xmin, xmax, ymin, ymax)
## coord. ref. : NAD83 / UTM zone 10N (EPSG:26910)
## source      : SS_Bathymetry.tif
```

```
## name        : SS_Bathymetry
## min value   :     -762.8359
## max value   :      190.5425
```

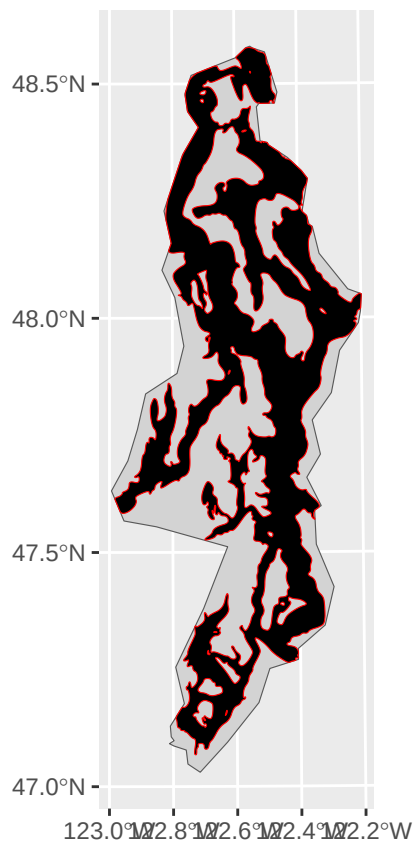
Creating a simple polygon with the bathymetry file

```
shore <- shoreline %>% dplyr::select(OBJECTID_1)

# crop raster to remove land values with shoreline
cropped_raster <- crop(ss_bath, shore)

cropped_raster2 <- crop(cropped_raster, studyarea)
#mask
masked_raster <- terra::mask(cropped_raster2, studyarea)
plot(masked_raster)
```




```
bath_poly <- st_as_sf(as.polygons(masked_raster, dissolve = TRUE),as_points = FALSE)

# Drop attributes, keeping only geometry
bath_poly_geom <- st_geometry(bath_poly)

# Union all geometries into a single polygon
single_polygon <- st_union(bath_poly_geom)
```

```r
# Convert to sf object
puget_poly <- st_sf(geometry = single_polygon)

polygon_buffered <- st_buffer(puget_poly, dist = 100)

polygon_simple <- st_simplify(polygon_buffered, dTolerance = 250)

polygon_smoothed <- smoothr::smooth(polygon_simple, method = "chaikin")

## clip it back to the the study area

puget_polygon1 <- st_intersection(polygon_smoothed, studyarea)


ggplot()+
  geom_sf(data = studyarea, fill = "lightgray")+
  geom_sf(data = puget_polygon1, fill = "black", color = "red")
```



save puget polygon. **After saving `puget_poly_clipped.gpkg`, I used QGIS to remove some small branching and random little pixels to simplify further. The simplified version that is used throughout the rest of analysis is called `puget_poly.gpkg`**

Reload simplified version

```r
puget_polygon <- st_read("GIS_COVS/GPKG/puget_poly.gpkg")
```

Creating Grids

Make Gridding

```r
cell_area <- units::as_units(8, "km^2") # target grid size

hex_grid <- st_make_grid(puget_polygon,square = T, cellsize = cell_area)

# create sf
grid_sf <- st_sf(geometry = hex_grid)
# clip to study area
grid_clipped <- st_intersection(grid_sf, puget_polygon)
# validate
grid_valid <- st_make_valid(grid_clipped)
```

Split into Single Polygons

I didn't want any multipolygons, so here I am breaking any multipolygons into single polygons, and then joining any neighboring polygons that are very small.

```r
overlap_pairs <- st_overlaps(grid_valid, grid_valid, sparse = FALSE)
sum(overlap_pairs)
```

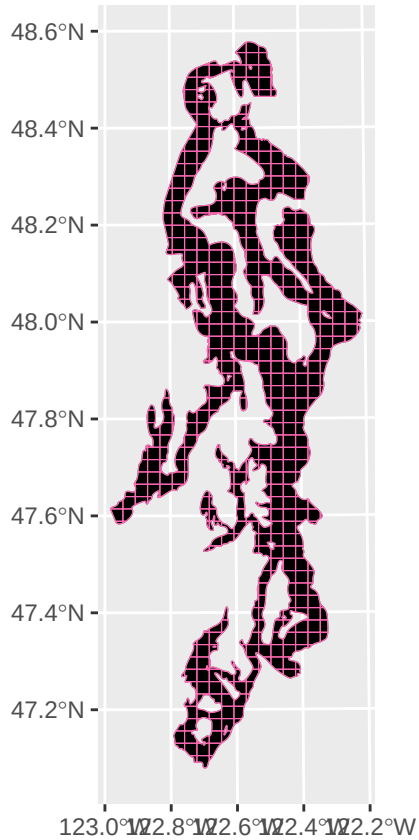
```
## [1] 0
```

```r
multipolygons <- grid_valid[st_geometry_type(grid_valid) == "MULTIPOLYGON", ]

overlap_pairs <- st_overlaps(multipolygons, multipolygons, sparse = FALSE)
sum(overlap_pairs)
```

```
## [1] 0
```

```r
parts <- st_as_sf(st_cast(multipolygons$geometry, "POLYGON"))

single <- grid_valid[st_geometry_type(grid_valid) == "POLYGON", ]


single <- single %>% st_geometry() %>%
  st_as_sf()

single_polys <- rbind(single,parts)

single_polys$id <- seq_len(nrow(single_polys))

single_polys <- single_polys %>%
  rename(geometry =x)
```

```
ggplot()+
  geom_sf(data = puget_polygon, fill="black") +
  geom_sf(data = single_polys, color="hotpink", fill=NA)
```

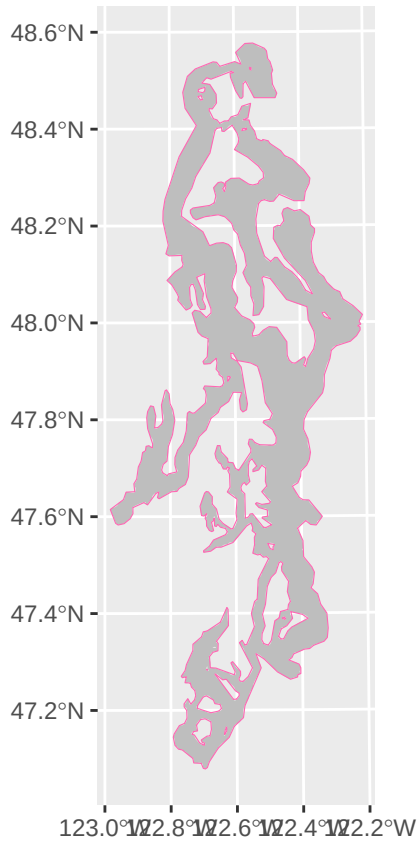


Merge small grids with neighbors

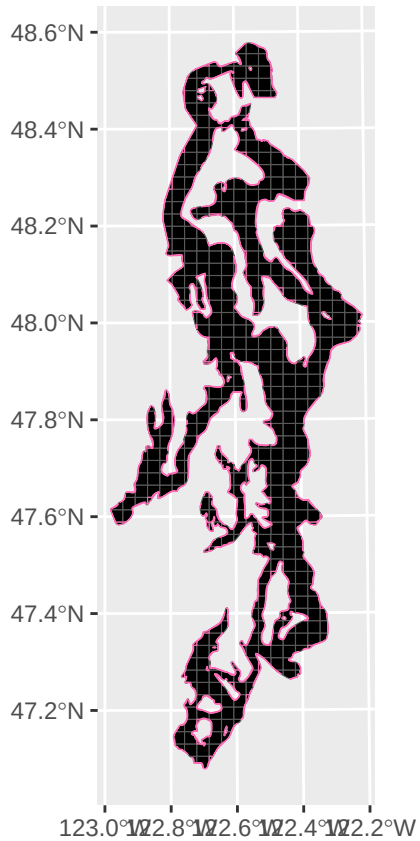
Filter to larger cells

After determining grids that are large enough, I am identifying "gaps" between the shoreline and the grids to determine where new grids need to be added.

```
poly_simplified <- puget_polygon %>%
  st_buffer(dist = 150) %>%
  st_simplify(dTolerance = 200,preserveTopology = TRUE) %>%
  st_buffer(dist = -50)



ggplot()+
  geom_sf(data = puget_polygon, fill="gray", color =NA) +
  geom_sf(data = poly_simplified, color="hotpink", fill=NA)
```




```r
## remove the small islands
area_thresh <- units::set_units(2, km^2)
p_dropped <- smoothr::fill_holes(poly_simplified, threshold = area_thresh)



threshold <- units::set_units(1000000, m^2)
big_cells <- single_polys %>% filter(st_area(geometry) >= threshold)

merged_hexes <- st_union(big_cells$geometry)

gaps <- st_difference(p_dropped, merged_hexes)
gaps <- st_cast(gaps, "POLYGON")

ggplot()+
  geom_sf(data = single_polys, fill="black") +
  geom_sf(data = gaps, color="hotpink", fill=NA)
```



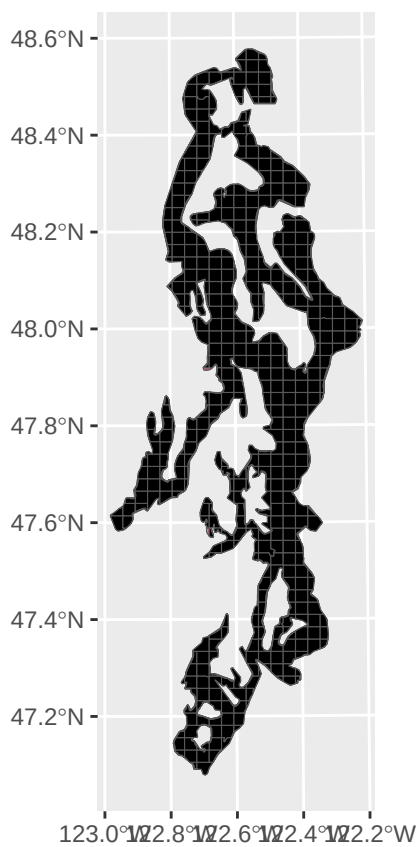
Process each gap individually

```
big_cells2 <- big_cells
for (i in seq_len(nrow(gaps))) {
  gap_geom <- gaps[i,]$geom  # Get individual gap polygon
  gap_sf <- st_as_sf(data.frame(geometry = gap_geom))
  # Find touching hexagons (neighbors that share an edge)
  touching_neighbors <- big_cells2[st_touches(big_cells2, gap_sf, sparse = FALSE), ]


  if (nrow(touching_neighbors) > 0) {
    # Calculate shared boundary length between the gap and each neighbor
    touching_neighbors <- touching_neighbors %>%
      rowwise() %>%
      mutate(shared_length = as.numeric(st_length(st_intersection(geometry,
                                                                  gap_sf)))) %>%
      ungroup()

    max_shared_length <- max(touching_neighbors$shared_length)

    # Find the neighbor with the longest shared boundary
    best_match_id <- touching_neighbors %>%
      filter(shared_length == max_shared_length) %>%
      mutate(min_area = min(st_area(geometry))) %>%
```

```r
      filter(st_area(geometry) == min_area) %>%
      mutate(centroid_dist = st_distance(st_centroid(gap_sf$geometry),
                                         st_centroid(geometry))) %>%
      slice_min(centroid_dist, n = 1) %>%
      pull(id)

    # Merge the gap into the best-matching hexagon
    new_geom <- st_union(big_cells2$geometry[big_cells2$id == best_match_id],
                         gap_sf$geometry)

    big_cells2$geometry[big_cells2$id == best_match_id] <- st_make_valid(st_cast(new_geom,
                                                                                 "POLYGON"))
  }
}


## check for multipolygons
nrow(big_cells[st_geometry_type(big_cells2) == "MULTIPOLYGON", ])
```

```
## [1] 0
```

```r
ggplot()+
  geom_sf(data = single_polys, fill="black") +
  geom_sf(data = gaps, fill="hotpink")+
  geom_sf(data = big_cells2, fill="black")
```

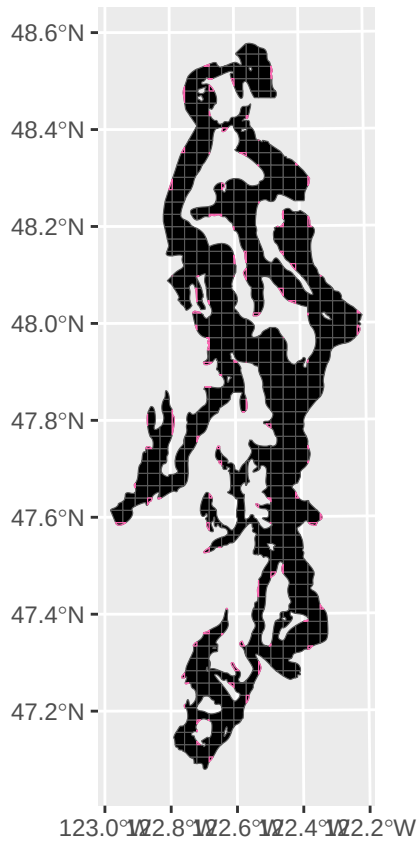


Merge small grids with neighbors

Filter to larger cells

Repeating the process until I have large enough grids with minimal gaps between shore and grids.

```
big_cells2 <- single_polys %>% filter(st_area(geometry) >= threshold)

merged_hexes <- st_union(big_cells2$geometry)

gaps <- st_difference(puget_polygon, merged_hexes)
gaps <- st_cast(gaps, "POLYGON")


ggplot()+
  geom_sf(data = single_polys, fill="black") +
  geom_sf(data = gaps, color="hotpink", fill=NA)
```



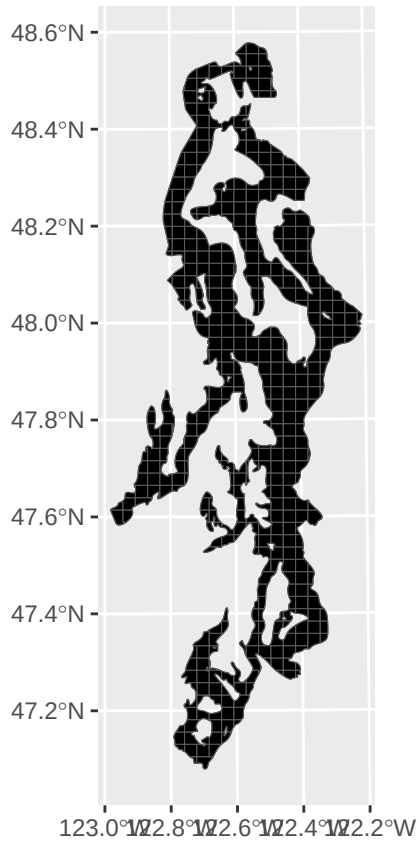
Process each gap individually

```
big_cells3 <- big_cells2
for (i in seq_len(nrow(gaps))) {
  gap_geom <- gaps[i,]$geom  # Get individual gap polygon
  gap_sf <- st_as_sf(data.frame(geometry = gap_geom))
```

```r
  # Find touching hexagons (neighbors that share an edge)
  touching_neighbors <- big_cells2[st_touches(big_cells2, gap_sf,
                                              sparse = FALSE), ]


  if (nrow(touching_neighbors) > 0) {
    # Calculate shared boundary length between the gap and each neighbor
    touching_neighbors <- touching_neighbors %>%
      rowwise() %>%
      mutate(shared_length = as.numeric(st_length(st_intersection(geometry,
                                                                  gap_sf)))) %>%
      ungroup()

    max_shared_length <- max(touching_neighbors$shared_length)

    # Find the neighbor with the longest shared boundary
    best_match_id <- touching_neighbors %>%
      filter(shared_length == max_shared_length) %>%
      mutate(min_area = min(st_area(geometry))) %>%
      filter(st_area(geometry) == min_area) %>%
      mutate(centroid_dist = st_distance(st_centroid(gap_sf$geometry),
                                         st_centroid(geometry))) %>%
      slice_min(centroid_dist, n = 1) %>%
      pull(id)

    # Merge the gap into the best-matching hexagon
    new_geom <- st_union(big_cells3$geometry[big_cells3$id == best_match_id],
                         gap_sf$geometry)

    big_cells3$geometry[big_cells3$id == best_match_id] <- st_make_valid(st_cast(new_geom,
                                                                                 "POLYGON"))
  }
}


## check for multipolygons
nrow(big_cells[st_geometry_type(big_cells3) == "MULTIPOLYGON", ])
```

```
## [1] 0
```

```r
ggplot()+
  geom_sf(data = single_polys, fill="black") +
  geom_sf(data = gaps, fill="hotpink")+
  geom_sf(data = big_cells3, fill="black")
```



Check & fix Overlap

```
overlap_pairs <- st_overlaps(big_cells3, big_cells3, sparse = FALSE)
sum(overlap_pairs)
```

```
## [1] 0
```

Confirm no multipolygons & overlap

```
any(!st_is_valid(big_cells3))
```

```
## [1] FALSE
```

```
nrow(big_cells3[st_geometry_type(big_cells3) == "MULTIPOLYGON", ])
```

```
## [1] 0
```

```
any(st_overlaps(big_cells3, big_cells3, sparse = F))
```

```
## [1] FALSE
```

Get Area & make valid

```
hex_final <- st_sf(geometry = big_cells3) %>%
  st_make_valid() %>%
  mutate(area = round(as.numeric(st_area(geometry))/1e6,2))


any(!st_is_valid(hex_final))
```

```
## [1] FALSE
```

```
nrow(hex_final[st_geometry_type(hex_final) == "MULTIPOLYGON", ])
```

```
## [1] 0
```

```
any(st_overlaps(hex_final, hex_final, sparse = F))
```

```
## [1] FALSE
```

```
hex_final$ID <- paste0("G",seq_len(nrow(hex_final)))
hex_final$numid <- seq_len(nrow(hex_final))

hex_final$id <- NULL
```

Ensure Neighboring works

```
neighbors <-spdep::poly2nb(hex_final, row.names=hex_final$numid,queen =F,
                           snap = 0.001)

all(names(neighbors) == hex_final$numid)
```

```
## [1] TRUE
```

```
adjacency_info <- spdep::nb2WB(neighbors)
```

Save

save the grids for analyses

```
st_write(hex_final,"GIS_COVS/OUTPUTS/grid4km_hex.gpkg", delete_layer = TRUE)
```